Sure! Here's an easy-to-understand guide on how to perform a dry run of the Extended-Bottom-Up-Cut-Rod algorithm. This guide is designed to help you understand and follow the steps when analyzing a problem similar to the one given.

Extended-Bottom-Up-Cut-Rod Algorithm Dry Run Guide

Problem Statement

Given a rod of length \(n \) and a list of prices \(p \) for each length, determine the maximum revenue \(r[n] \) obtainable and the lengths of pieces \(s \) to achieve that revenue.

Input Example

- \(n = 6 \)

- \(p = \langle 4, 4, 13, 40, 43, 49 \rangle \)

Step-by-Step Guide

1. **Initialize Arrays**:

   - Create two arrays \(r \) and \(s \) of size \(n+1 \) and set all elements to 0.

   - Set \(r[0] = 0 \).

   ```
   r = [0, 0, 0, 0, 0, 0, 0]
   s = [0, 0, 0, 0, 0, 0, 0]
   ```

2. **Iterate Over Each Length \(j \) from 1 to \(n \)**:

- For each j, initialize q to negative infinity ($-\infty$).
- Iterate over each possible cut length i from 1 to j:
- Calculate potential revenue: $p[i-1] + r[j-i]$.
- If this potential revenue is greater than current q:
- Update q with this new revenue.
- Update $s[j]$ with i.

3. **Update $r[j]$**:
- After the inner loop, set $r[j]$ to q.

Example Dry Run

- **For $j = 1$**:
- Initialize $q = -\infty$.
- **Cut at $i = 1$**: $p[0] + r[0] = 4 + 0 = 4$
- Update $q = 4$, $s[1] = 1$.
- Set $r[1] = 4$.

```
r = [0, 4, 0, 0, 0, 0, 0]
s = [0, 1, 0, 0, 0, 0, 0]
```

- **For $j = 2$**:
- Initialize $q = -\infty$.
- **Cut at $i = 1$**: $p[0] + r[1] = 4 + 4 = 8$
- Update $q = 8$, $s[2] = 1$.
- **Cut at $i = 2$**: $p[1] + r[0] = 4 + 0 = 4$ (no update since $8 > 4$)

- Set \(r[2] = 8 \).

```
r = [0, 4, 8, 0, 0, 0, 0]
s = [0, 1, 1, 0, 0, 0, 0]
```

- **For \(j = 3 \)**:
  - Initialize \(q = -\infty \).
  - **Cut at \(i = 1 \)**: \(p[0] + r[2] = 4 + 8 = 12 \)
    - Update \(q = 12 \), \(s[3] = 1 \).
  - **Cut at \(i = 2 \)**: \(p[1] + r[1] = 4 + 4 = 8 \) (no update since 12 > 8)
  - **Cut at \(i = 3 \)**: \(p[2] + r[0] = 13 + 0 = 13 \)
    - Update \(q = 13 \), \(s[3] = 3 \).
  - Set \(r[3] = 13 \).

```
r = [0, 4, 8, 13, 0, 0, 0]
s = [0, 1, 1, 3, 0, 0, 0]
```

- **For \(j = 4 \)**:
  - Initialize \(q = -\infty \).
  - **Cut at \(i = 1 \)**: \(p[0] + r[3] = 4 + 13 = 17 \)
    - Update \(q = 17 \), \(s[4] = 1 \).
  - **Cut at \(i = 2 \)**: \(p[1] + r[2] = 4 + 8 = 12 \) (no update since 17 > 12)
  - **Cut at \(i = 3 \)**: \(p[2] + r[1] = 13 + 4 = 17 \) (no update since 17 = 17)
  - **Cut at \(i = 4 \)**: \(p[3] + r[0] = 40 + 0 = 40 \)

  - Update \(q = 40 \), \(s[4] = 4 \).

  - Set \(r[4] = 40 \).

  ```
  r = [0, 4, 8, 13, 40, 0, 0]
  s = [0, 1, 1, 3, 4, 0, 0]
  ```

- **For \(j = 5 \)**:

  - Initialize \(q = -\infty \).

  - **Cut at \(i = 1 \)**: \(p[0] + r[4] = 4 + 40 = 44 \)

    - Update \(q = 44 \), \(s[5] = 1 \).

  - **Cut at \(i = 2 \)**: \(p[1] + r[3] = 4 + 13 = 17 \) (no update since 44 > 17)

  - **Cut at \(i = 3 \)**: \(p[2] + r[2] = 13 + 8 = 21 \) (no update since 44 > 21)

  - **Cut at \(i = 4 \)**: \(p[3] + r[1] = 40 + 4 = 44 \) (no update since 44 = 44)

  - **Cut at \(i = 5 \)**: \(p[4] + r[0] = 43 + 0 = 43 \) (no update since 44 > 43)

  - Set \(r[5] = 44 \).

  ```
  r = [0, 4, 8, 13, 40, 44, 0]
  s = [0, 1, 1, 3, 4, 1, 0]
  ```

- **For \(j = 6 \)**:

  - Initialize \(q = -\infty \).

  - **Cut at \(i = 1 \)**: \(p[0] + r[5] = 4 + 44 = 48 \)

    - Update \(q = 48 \), \(s[6] = 1 \).

  - **Cut at \(i = 2 \)**: \(p[1] + r[4] = 4 + 40 = 44 \) (no update since 48 > 44)

- **Cut at \(i = 3 \)**: \(p[2] + r[3] = 13 + 13 = 26 \) (no update since 48 > 26)

- **Cut at \(i = 4 \)**: \(p[3] + r[2] = 40 + 8 = 48 \) (no update since 48 = 48)

- **Cut at \(i = 5 \)**: \(p[4] + r[1] = 43 + 4 = 47 \) (no update since 48 > 47)

- **Cut at \(i = 6 \)**: \(p[5] + r[0] = 49 + 0 = 49 \)

  - Update \(q = 49 \), \(s[6] = 6 \).

- Set \(r[6] = 49 \).

```
r = [0, 4, 8, 13, 40, 44, 49]

s = [0, 1, 1, 3, 4, 1, 6]
```

Final Arrays

- \(r = [0, 4, 8, 13, 40, 44, 49] \)

- \(s = [0, 1, 1, 3, 4, 1, 6] \)

Interpretation of Results

- \(r[n] = 49 \):

The maximum revenue obtainable from a rod of length 6.

- \(s \) array tells the lengths of the first cut to make for each rod length to achieve this maximum revenue.

Use this guide to practice running through the algorithm step by step, ensuring you understand how each decision is made at every step. This understanding will be crucial during an exam.